arXiv:2606.17463v2 [cs.CV] 7 Sep 2026

WeaveLA: Event Driven Cross-Subtask Latent Memory Weaving for Repetition-Aware Robot Manipulation

Shoujing Zhu[1,*,‡] **Zhenyang Liu**[1,3,*] **Fungmiu Wong**[1,*] **Jiafeng Wang**[1] **Bo Yue**[2]
Guiliang Liu[2,4] **Simo Wu**[1] **Xiangyang Xue**[1,†] **Taiping Zeng**[1,†]

[1] Fudan University [2] School of Data Science, The Chinese University of Hong Kong, Shenzhen
[3] Shanghai Innovation Institute [4] Shenzhen Loop Area Institute

{zhusj25, liuzy24, fmwang23, jfwang25}@m.fudan.edu.cn, boyue@link.cuhk.edu.cn,
liuguiliang@cuhk.edu.cn, wusimo19911125@gmail.com
{xyxue, zengtaiping}@fudan.edu.cn

[*] Equal contribution [‡] Project Lead [†] Corresponding author

Abstract

Vision-Language-Action (VLA) policies have achieved remarkable single-step manipulation, yet they remain brittle precisely where each stage depends on what was just completed. The core issue is structural: short-window VLAs lack an explicit channel for rouxting information across sub-task boundaries, and existing memory-augmented variants either write at every frame, retrieve from demonstration-time stages, or fire at sub-goal events without performing an explicit sub-task-to-sub-task hand-off into the action expert. We identify the sub-goal completion event as the natural temporal unit for cross-subtask memory hand-off, and present WeaveLA (Weave Latent memory for Vision-Language-Action policies), a cross-subtask memory interface that, on top of a frozen VLA backbone, compresses each completed segment into latent tokens via query-driven attention pooling and routes them directly into the action-generation path of the next sub-task. This event-triggered, action-side design preserves the base policy's short-window interface while adding a lightweight cross-subtask channel. Through stratified evaluation on RoboMME with a $\pi_{0.5}$ backbone, WeaveLA's gains land exactly where the channel is needed: on the hardest repetition slice (SwingXtimes, $N{=}3$), success rises from 0% to 47.8%, while single-execution episodes remain unchanged. Per-episode paired analysis confirms the gains are confined to tasks whose causal structure requires cross-subtask information.

1 Introduction

Robotic manipulation has witnessed remarkable progress in recent years, largely propelled by the scaling of Vision-Language-Action (VLA) policies. Models such as RT-2 [5], OpenVLA [17], π_0/$\pi_{0.5}$ [4, 29], and the GR00T N1.x family [25, 26] achieve impressive single-step manipulation by conditioning on the current observation [45], proprioceptive state, and natural-language instruction within a short temporal window. They reliably grasp objects, press buttons, and place items in compact contexts [28, 34].

However, a large fraction of real-world manipulation work is not single-step but *repetitive*: an assembly cell repeats the same insertion motion, a packaging station places N items into a bin until full, a quality-control fixture stops the cube only after the right number of swings. The defining property of such episodes is not their length but their *cross-subtask dependency* — what to do at the current stage depends on what was just completed at the previous one. Current short-window VLAs

Preprint.

collapse precisely here: not because they cannot execute the primitive, but because once one sub-task ends the next is launched without any compact summary of what just happened. There is no explicit channel for routing information across sub-task boundaries, and the policy has no persistent signal that the previous stage has been completed.

Recent memory-augmented VLAs attempt to address this by adding temporal context projection to the policy — through multi-frame summaries, visual-trace overlays, growing memory banks, or retrieval over demonstration-time stages. These methods differ in *what* they store and *where* they consume it, but they almost all share the same default *when*: write at every frame, over a sliding window, or via demonstration-stage retrieval. A more recent line [51, 7, 49, 50, 13] fires writes at sub-goal events but stops short of a true cross-subtask hand-off that matters for repetitive manipulation — none of these triggers delivers a self-contained summary to the next sub-task's action expert. We provide a detailed comparison in Section 2.

We identify a key principle: **the sub-goal completion event is the natural temporal unit at which cross-subtask information becomes available, and the next stage begins to depend on it.** A per-frame writer must implicitly discover boundaries from dense observations and pays a per-step memory cost; a demo-stage retrieval mechanism cannot key onto rollout-time progress. Writing precisely at the sub-goal event aligns the information hand-off with the causal structure of repetitive tasks — the moment one stage ends is exactly when the next stage needs to know what was accomplished.

Based on this principle, we present **WeaveLA**, a cross-subtask latent memory weaving interface for repetition-aware robot control. At each sub-goal completion event, a Query-driven Memory Weaver uses 8 learnable query tokens [21] to compress the just-completed segment into a compact latent (N=8 tokens, d=2048) via single-step attention pooling, and routes this latent directly into the *action-generation path* of the next sub-task — not the language prompt, not the visual context. The action-side injection ensures cross-subtask information reaches the motor-execution level without being diluted through the VLM backbone. We instantiate the interface on $\pi_{0.5}$ via AdaRMS modulation of the Gemma action expert. The pretrained backbone remains frozen, and only a lightweight Weaver module and an action-side memory modulation are trained, preserving the base policy's competence on non-repetitive tasks.

Through stratified evaluation on RoboMME, WeaveLA delivers gains that land exactly where the cross-subtask channel is needed. On the hardest repetition slice — SWINGXTIMES at N=3, where the policy must keep track of how many back-and-forth motions have already happened — success rises from 0% to 47.8%. Critically, single-execution episodes (N=1) remain at $\sim 100\%$ in both conditions, confirming that the mechanism activates only where the channel is required rather than acting as a generic capacity boost. Per-episode paired analysis on 800 matched episodes confirms the imbalance is confined to tasks whose causal structure demands cross-subtask information.

In summary, our contributions are:

1. We identify the sub-goal completion event as the natural temporal unit for cross-subtask memory hand-off, and argue the hand-off should deliver a self-contained summary of the just-completed sub-task directly to the next sub-task's action expert — a *when*-axis design dimension under-explored by per-frame and demonstration-retrieval memory mechanisms.
2. We propose WeaveLA, a query-driven attention-pooling memory interface that writes 8 latent tokens at each sub-goal boundary and feeds them directly into the action-generation path. The pretrained VLA backbone is preserved untouched; only a small Weaver module and an action-side projection are trained.
3. Through repetition-stratified, difficulty-stratified, and per-episode paired analyses on RoboMME, we provide mechanism-aligned evidence: WeaveLA's gains land where the cross-subtask channel is needed and stay quiet where it is not, with the strongest single result moving SWINGXTIMES N=3 from 0% to 47.8%.

2 Related work

2.1 Memory-augmented VLA policies

Memory-augmented VLA policies differ along three axes: *what* is stored, *where* it is consumed, and *when* the write is triggered. Prior work stores dense visual histories or overlays [19, 43, 10, 44],

semantic traces or retrieved episodes [36, 32, 22], and recurrent/query summaries [30, 20]; these memories are consumed through prompt/input augmentation [22, 16, 24], planner-level memory [32, 42], or action-path modulation. WeaveLA targets the last setting with a compact event-level latent routed directly into the next sub-task's action expert.

Along the *when* axis — the write trigger — existing methods either write at every frame or retrieve from demonstration-time stages, and a more recent line [51, 7, 49, 50, 13], alongside a longer history of skill-segment IL [18, 53, 52], fires at sub-goal/segment events but never delivers a self-contained hand-off to the next sub-task's action expert. WeaveLA identifies the sub-goal completion event as the natural write trigger and contributes on the hand-off side, routing a compressed per-segment latent directly into the action expert.

2.2 Query-bottleneck memory modules

Learnable query bottlenecks are a general mechanism for compressing variable-length context into a fixed-size latent set. They bridge frozen vision and language modules in BLIP-2's Q-Former [21], support hierarchical or cross-segment context in robot and sequence models [42, 6], and appear in imitation-learning abstractions that compress variable-length sub-segments [18]. The trigger–weaver factorisation we adopt is directly inspired by MemGen [46], which couples a learned trigger with a learned latent weaver for text-LLM reasoning; WeaveLA retargets that factorisation to embodied control by replacing sentence boundaries with sub-goal completion events and routing the latent into the action-generation path of a frozen VLA. Thus the query bottleneck summarises a *just-completed sub-task segment* for a downstream *action expert*, rather than a single image for language generation. We compare single-step attention pooling with a Q-Former-style decoder stack and find that the simpler attention-pooling extractor is more stable in the rollout-time, action-grounded regime (Section 4.4).

2.3 Task-structure priors and repetition counting

Repetitive manipulation is not a niche academic problem: industrial work such as assembly, packaging, inspection, polishing, and kitting often requires executing the same primitive a prescribed number of times, where the hard part is knowing should next move is repetitive. Control-side task-structure methods make this state explicit through reward machines [37] or LTL-conditioned policies [38], which expose automaton states at sub-task boundaries. WeaveLA shares the view that boundaries are informationally privileged, but replaces hand-specified symbolic state with a learned latent shaped end-to-end by action prediction. Perception-side repetition work instead focuses on recognising or counting repeated activity in video [9, 15, 31, 27, 41]. These works address the perceptual question of *recognising* repetitions; WeaveLA addresses the complementary control question: once a sub-task has just been completed, how should that event be routed into the policy so the next stage is conditioned on it?

3 WeaveLA architecture

3.1 Overview

WeaveLA targets manipulation episodes that decompose into ordered sub-tasks $\mathcal{E} = (\tau_0, \tau_1, \ldots, \tau_{K-1})$. A standard VLA policy predicts an action chunk [48, 39] a_t from the current observation o_t, proprioceptive state s_t, and language instruction ℓ. WeaveLA adds one cross-subtask memory channel: at the completion event of τ_{k-1}, a compact latent m_{k-1} summarises the just-completed segment and conditions the action expert throughout τ_k. The interface is action-side — the language prompt and visual context of the base policy are unchanged — and the pretrained backbone is frozen except for low-rank adapters and a small Weaver module. Figure 2 illustrates a hybrid memory interface that combines history tokens F_t, representing preceding observations, with the event tokens m_{k-1} produced by the Weaver. The two sequences are concatenated along the token dimension, $M_t = [F_t \| m_{k-1}]$, retaining both historical context and a compact summary of the latest completed sub-task. In each action-expert block, action hidden states supply the queries and memory tokens supply the keys and values for memory cross-attention; the resulting context generates the scale and shift of Memory RMSNorm before the FFN. Thus, history and event information are fused through action-dependent attention and feature modulation.

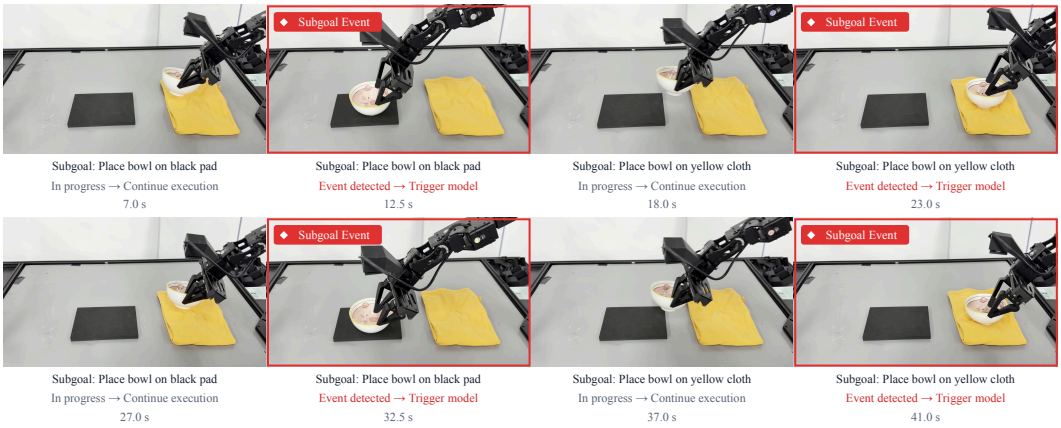

Figure 1: **Event-driven memory hand-off in a repetitive manipulation task.** The robot repeatedly transfers a bowl back and forth between a black pad and a yellow cloth, combining repeated placement with the arm's back-and-forth swinging motion. The eight frames show two successive round trips, alternating ongoing execution with sub-goal completion events. Each highlighted event marks a completed placement and triggers the Weaver to summarise that segment and update the event memory for the next sub-task; between events, execution continues with the latest event memory.

3.2 Sub-goal-event trigger

A central design choice distinguishes WeaveLA from per-frame [19] and demo-stage retrieval [22] memory: the memory write is triggered only at sub-goal completion events during rollout. For simulation experiments, sub-goal boundaries are provided by the simulator, isolating the memory mechanism from the boundary-detection problem. A sub-goal completion event e_k is associated with segment τ_{k-1} (frames between e_{k-1} and e_k); between events, the action expert continues to be conditioned on the most recent m_{k-1}. Figure 1 illustrates this schedule in a repetitive manipulation task: the robot repeatedly moves a bowl back and forth between a black pad and a yellow cloth, placing it at each destination as the arm swings between the two locations. Execution frames alternate with sub-goal completion events across successive repetitions.

The sub-goal event aligns the memory write with the natural unit at which the next stage depends on the previous one — the same temporal abstraction that the options framework [35, 2] formalises through option termination conditions. We deliberately do not assume the resulting latent encodes any particular structured quantity; whatever it encodes is shaped by the downstream action-prediction loss.

3.3 Query-driven memory weaver

For a completed segment τ_{k-1}, we extract perceptual features from the segment's frames: visual tokens projected from the frozen vision encoder of the underlying VLA, and projected proprioceptive state tokens. These are concatenated and layer-normalised [1] into a multimodal feature sequence $H \in \mathbb{R}^{B\times L\times d_h}$. A fixed set of $N{=}8$ learnable query latents $Q \in \mathbb{R}^{N\times d_h}$ summarises H via a single attention-pooling step [40]:

$$A = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_h}}\Big), \qquad m_{k-1} = W_{\text{out}}\,(A\,V + Q), \tag{1}$$

where $K{=}V{=}H$ and W_{out} is a learned linear projection to the memory width $d{=}2048$. We use $d_h{=}1024$ for the latent space and $d{=}2048$ at the output. The query residual $+Q$ preserves a non-zero gradient path to the queries when the attention pattern is sharp. The memory is segment-local: each m_{k-1} is computed only from the perceptual features of τ_{k-1}, and we do not feed m_{k-2} back into the encoder. This independent-per-hop design also matches what we found training-stable; see Section 3.5 and Appendix A.11 for the staged-training analysis.

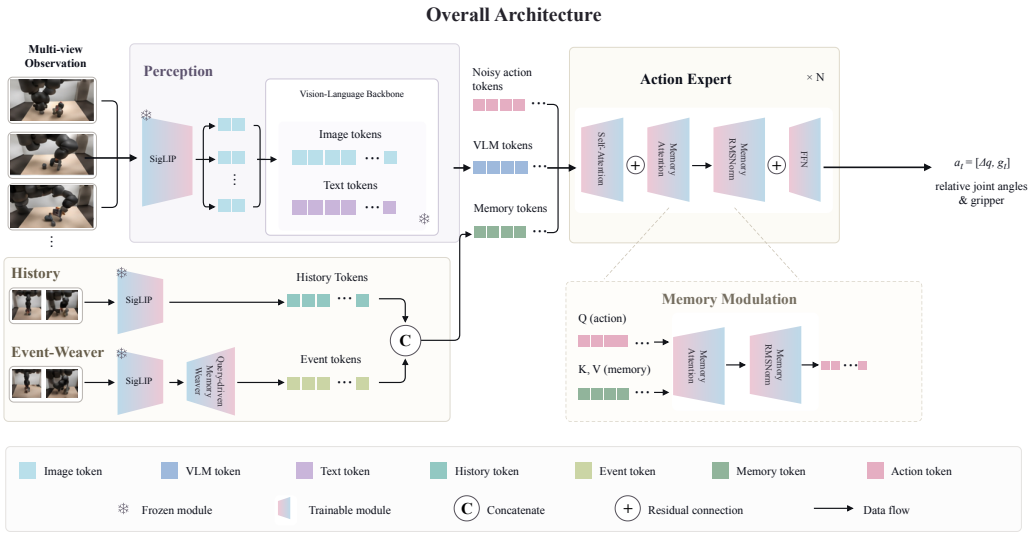

Figure 2: **WeaveLA architecture with history–event memory fusion.** Multi-view observations and text are encoded by the perception pathway, while the History and Event-Weaver branches provide complementary history and event tokens. The concatenation operator (C) joins these sequences into memory tokens for the action expert. In each repeated block, action features query the memory through Memory Attention, and the resulting context modulates those features through Memory RMSNorm before the FFN. The inset details the action-query/memory-key–value interaction. The action expert outputs relative joint angles and a gripper command. Snowflakes denote frozen modules, and token colours distinguish the information streams.

3.4 Action-side memory consumption

$\pi_{0.5}$ instantiation. Each action-expert Gemma [11, 12] block in $\pi_{0.5}$ [29] already contains two timestep-conditioned AdaRMS modules (one before self-attention, one before the FFN). WeaveLA leaves both untouched and *inserts* a third, memory-conditioned AdaRMS step between the self-attention residual and the existing pre-FFN AdaRMS, so the base time-conditioning pathway is preserved. Given the concatenated history and event memory $M = M_t = [F_t \| m_{k-1}] \in \mathbb{R}^{B \times L_M \times d_m}$, where L_M denotes the total number of memory tokens, we project it to the action-expert width, $\tilde{M} = MW_m \in \mathbb{R}^{B \times L_M \times d_{\text{act}}}$. For action hidden states $X_\ell \in \mathbb{R}^{B \times T \times d_{\text{act}}}$ at layer ℓ, the memory context (with RoPE [33] on Q/K and RMSNorm [47] pre-norm) is

$$\begin{aligned} Q_\ell &= \mathrm{RoPE}(\mathrm{RMSNorm}(X_\ell)W_\ell^Q), \\ K_\ell &= \mathrm{RoPE}(\mathrm{RMSNorm}(\tilde{M})W_\ell^K), \qquad V_\ell = \mathrm{RMSNorm}(\tilde{M})W_\ell^V, \\ C_\ell &= \mathrm{ConcatHeads}\Big[\mathrm{softmax}\Big(\frac{Q_\ell K_\ell^\top}{\sqrt{d_{\text{head}}}} + \mathcal{M}\Big) V_\ell\Big] W_\ell^O, \end{aligned} \tag{2}$$

where $\mathcal{M}$ masks padded memory slots. The token-wise context then produces scale and shift parameters for the memory-conditioned AdaRMS modulation,

$$[\gamma_\ell, \beta_\ell] = C_\ell W_\ell^\Delta + b_\ell^\Delta, \qquad X'_\ell = \frac{X_\ell}{\sqrt{\mathrm{mean}(X_\ell^2) + \varepsilon}} \odot (1 + \gamma_\ell) + \beta_\ell. \tag{3}$$

This design keeps memory modulation lightweight: it only adjusts the AdaRMS scale/shift, masks invalid memory tokens, and starts near the unmodulated RMSNorm behaviour via near-zero initialisation.

3.5 Staged training strategy

Training proceeds in three stages, separating action grounding, memory warmup, and full integration. Stage 0 performs task-set adaptation with the vision encoder frozen and only the flow-matching action loss active — this establishes a competent short-window policy on the target task family before any

memory mechanism is introduced. Stage 1 (K=2 multi-subtask windows) introduces the Weaver and the memory-conditioning pathway, but trains them only against the action loss; this gives the memory pathway enough time to stabilise before semantic auxiliary objectives are added. Stage 2 (variable $K \in \{2, 3, 4\}$) enables the full objective with semantic alignment and contrastive auxiliaries.

We also tested a single merged stage that combines all three objectives from the beginning. Despite stable action and alignment loss curves, the raw memory latent norm decayed monotonically across training (a hidden representation collapse not visible in the loss curves), and downstream evaluation success collapsed to near-zero. Action-grounded warmup before semantic objectives is therefore not an arbitrary curriculum choice but a necessary stabilisation of the memory pathway. We provide the diagnostic `weaver_latent_norm` metric and the merged-stage failure curve in Appendix A.11 (Figure 7).

3.6 Training objective and computational footprint

Action loss. The primary objective is the base policy's flow-matching action loss [23]. Given a clean action chunk a and a noise sample $\epsilon \sim \mathcal{N}(0, I)$, we sample $t \sim \mathrm{Beta}(1.5, 1)$ on $(0, 1]$ (diffusion-style convention: t=1 is noise, t=0 is the clean action) and form $a_t = t\,\epsilon + (1 - t)\,a$. The action expert predicts the velocity field $v_\theta(a_t, t \mid o_t, s_t, \ell, m_{k-1})$ conditioned on the observation, state, instruction, and Weaver memory m_{k-1}, and we minimise

$$\mathcal{L}_{\text{action}} \;=\; \mathbb{E}_{t,a,\epsilon} \left\| v_\theta(a_t, t \mid o_t, s_t, \ell, m_{k-1}) - (\epsilon - a) \right\|_2^2. \tag{4}$$

Sub-goal annotations are training-time supervision only. In Stage 2 we additionally use a memory–text alignment loss $\mathcal{L}_{\text{align}}$ and a memory contrastive loss $\mathcal{L}_{\text{ctr}}$, weighted at 0.05 and 0.02 respectively. The total loss is $\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda_{\text{align}}\mathcal{L}_{\text{align}} + \lambda_{\text{ctr}}\mathcal{L}_{\text{ctr}}$. Both auxiliaries operate on a tokenised description of the just-completed sub-goal — $\mathcal{L}_{\text{align}}$ matches the Weaver latent to a text encoder embedding of the sub-goal description, and $\mathcal{L}_{\text{ctr}}$ pulls together latent–text pairs from the same trajectory while pushing apart pairs from different trajectories. *This sub-goal text is consumed only during training.* At inference time the policy receives only the original episode-level instruction; sub-goal completion events drive the memory write, but no sub-goal text is fed to the model. The auxiliaries serve to shape the latent space during training; they do not change the model's input interface at deployment.

Computational footprint. Trainable parameters consist of three groups: (i) PaliGemma [3] LoRA [14] adapters in the $\pi_{0.5}$ backbone, (ii) the Weaver query latents and projection matrices, and (iii) the action-side memory cross-attention and AdaRMS scale/shift projections. In total the trainable parameter count is approximately 46M, on top of the frozen ~ 3.4B-parameterr base policy — about 1.4% of the base model. Because the memory write is triggered only at sub-goal boundaries rather than every frame, the runtime cost of the memory channel during rollout is negligible compared to dense per-frame memory mechanisms; a per-frame writer with the same latent size would incur a memory-extraction forward pass at every observation step.

4 Experiments

4.1 Experimental setup

Benchmark. We evaluate on RoboMME [8], which reports per-task success rates over 50 episodes per task across 16 manipulation tasks split into Easy / Medium / Hard sub-pools. The two repetition tasks (PICKXTIMES, SWINGXTIMES) sample episodes whose natural-language instruction encodes a target count N specifying *the number of sub-goals to be executed in that episode* (e.g., N=3 means three pick-and-place repetitions before the stop button is pressed). Unless context indicates the memory bottleneck width (where N=8 is fixed throughout, except in the width ablation of Appendix A.10), N refers to this sub-goal count. Main-text figures report results on the six tasks used for $\pi_{0.5}$ training; the full 16-task breakdown is in Appendix A.1.

Why SWINGXTIMES and STOPCUBE are hard. Both tasks share a defining property absent from non-repetitive RoboMME tasks: *the correct action at the current stage cannot be inferred from the current observation alone*. SWINGXTIMES [8] requires the policy to swing a cube back and forth

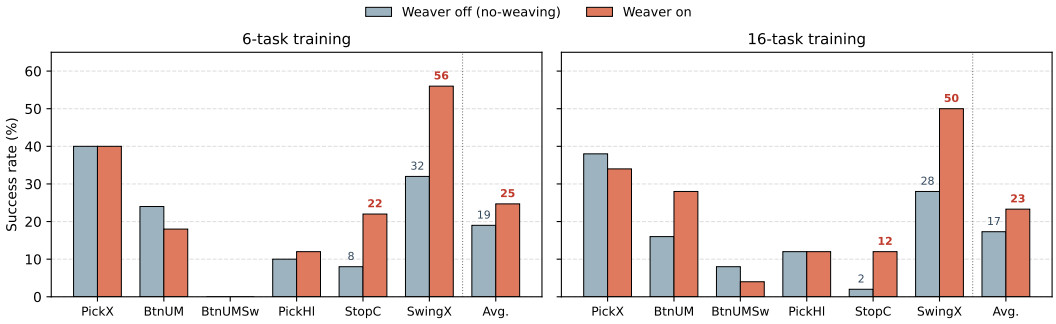

Figure 3: **Aggregate $\pi_{0.5}$+AP results on the six trained tasks** (success ↑, %, 50 episodes). Improvements concentrate on StopC/SwingX at both training scales. "Weaver off (no-weaving baseline)" uses our framework with the memory-weaving pathway disabled, sharing training data, optimiser with the corresponding Weaver-on run.

N times between two targets and then press a stop button. The instruction encodes N, but visually the scene at swing 1 and swing 3 is indistinguishable — the cube and the targets occupy the same positions. The policy must carry a persistent signal of how many swings have already occurred. StopCube [8] requires the policy to remain static and press a stop button only after the cube has reached a target the prescribed number of times; here too the visual scene is ambiguous between the "not yet" and the "now" moments, and the decision to press is determined by the trajectory of the previous stages. Tasks like ButtonUnmask or PickHighlight, by contrast, can be solved by reactively responding to the current observation. The repetition / dependent-stop tasks are therefore a clean stress test for whether a policy can route information across sub-task boundaries; failures on these tasks isolate the cross-subtask channel as the bottleneck.

Backbones and instantiations. The main results in Sections 4.2–4.3 use a $\pi_{0.5}$ backbone with attention-pooling Weaver (denoted "$\pi_{0.5}$+AP"). For RoboMME we report two training scales: 6-task (PickXtimes, ButtonUnmask, ButtonUnmaskSwap, PickHighlight, StopCube, SwingXtimes) and 16-task (all RoboMME tasks); all four $\pi_{0.5}$+AP runs (Weaver-on/off $\times$ 6-task/16-task) share training stages, differing only in the Weaver mechanism and the training-task set. We further probe two orthogonal axes of the design: an extractor study (single-step attention pooling vs. Q-Former-style multi-layer decoder, "$\pi_{0.5}$+QF"; Section 4.4) and a trigger-source study (oracle vs. model-internal latent-shift detector with the extractor fixed at attention pooling; Section 4.5). Full hyper-parameters are in Appendix A.12.

4.2 Aggregate results

Figure 3 reports per-task success on the six trained tasks for $\pi_{0.5}$+AP. The aggregate average improves from 19.0% (Weaver off, 6-task) to 24.7% (Weaver on, 6-task), with the largest improvements on SwingXtimes ($32\% \rightarrow 56\%$) and StopCube ($8\% \rightarrow 22\%$) — the two tasks identified above as requiring cross-subtask information. At the 16-task scale the same framework improves from 17.3% to 23.3%, again driven by the repetition / dependent-stop tasks. These aggregates are a starting point; the following analyses test whether the gains land where the cross-subtask channel is actually needed. The full per-task numerical table is in Appendix A.2.

4.3 Repetition-stratified analysis

The aggregate numbers understate where the mechanism actually engages. Figure 4 stratifies per-episode outcomes on the two repetition tasks by the instruction-encoded sub-goal count N. With $N{=}1$ (a single sub-goal, no cross-subtask dependency), both Weaver-on and Weaver-off succeed at $\sim 100\%$, as expected: a single execution does not depend on a previously completed sub-goal. As soon as $N{\geq}2$, the gap opens, and it widens with N.

The headline finding lives in the hardest sub-population. On SwingXtimes at $N{=}3$ (three sub-goals required), Weaver-off succeeds in 0% of episodes; **Weaver-on succeeds in** 47.8%. The same setting at the 16-task training scale moves from 4% to 30%. This is the slice where the cross-subtask channel

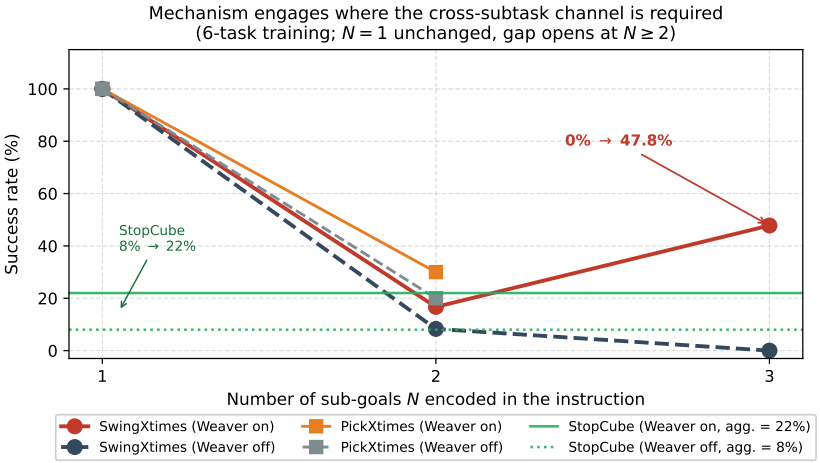

Figure 4: **Repetition-stratified success on SWINGXTIMES and PICKXTIMES** (success ↑, %; 6-task training; N = number of sub-goals encoded in the instruction). Annotations in the figure are rounded to integer percentages; the underlying value at $N{=}3$ is 47.8% (rounded to 48%). Both conditions reach $\sim$ 100% at $N{=}1$; the gap opens at $N{\geq}2$. STOPCUBE aggregate Weaver-on/off levels are overlaid as horizontal references.

is most critical: the visual scene is ambiguous between an unfinished and a near-finished swing trajectory, and only a persistent signal from previously completed sub-goals can determine the correct action.

Pooling across both repetition tasks at the 6-task scale, Weaver-on succeeds on 24.6% of $N{\geq}2$ episodes against 7.2% for Weaver-off — a $3.4\times$ relative improvement. At the 16-task scale the same pool gives 17.4% versus 5.8% ($3.0\times$ relative). The sign and shape are stable across data scales. The neutrality at $N{=}1$ is as informative as the gap at $N{\geq}2$: a mechanism that improved $N{=}1$ would betray that its gains came from incidental capacity rather than from the cross-subtask channel. The STOPCUBE reference levels overlaid in Figure 4 show that the same mechanism-localisation pattern holds on the dependent-stop task as well, even though it does not have an explicit N-stratification axis. The full N-stratified table is in Appendix A.3.

Repetition stratification gives the cleanest view of where the mechanism engages, but two further analyses confirm the same pattern. *Difficulty stratification* on the six trained tasks shows the gain concentrates on Hard episodes ($1.4\% \rightarrow 12.5\%$), while Easy and Medium episodes change only marginally. *Per-episode paired analysis* on the same matched 50 episodes per task confirms the localisation directly: counting episodes that only one condition solves, Weaver-on uniquely succeeds on 13 episodes vs. 1 for Weaver-off on SWINGXTIMES, and 8 vs. 1 on STOPCUBE; on every other task the two counts differ by at most 3. Both stratifications mirror the N-stratified picture: the cross-subtask channel engages exactly on the tasks whose causal structure requires it. Full tables are in Appendix A.4 (difficulty) and Appendix A.5 (paired contingency).

StopCube as a what/where/when ablation. STOPCUBE is the canonical *count, time-critical* task in RoboMME [8]: the policy must press the button at the N-th moment a moving cube reaches the target. We read STOPCUBE as a one-axis-at-a-time ablation around our anchor **WeaveLA at 22.0%**; Figure 5 shows what each one-axis swap costs. Varying *what* is stored, every alternative collapses to single digits because the substituted representation cannot carry the per-event count signal: recurrent state (TTT, RMT) compresses count into a diffuse hidden state, sparse perceptual (TokenDrop+Modul) drops the slow continuous evidence, non-oracle symbolic (GroundSG+QwenVL) compounds subgoal-prediction errors across visits, and the rich-perceptual-but-count-blind prior baselines MemER [32] and SAM2Act+ [10] target keyframe retrieval / waypoint prediction rather than count tracking, figure 5 showing baseline numbers from Table 12 of Dai et al. [8]. Only a dense per-frame buffer (FrameSamp+Modul, 42.0%) beats us. Varying *where* or *when*, neither axis explains that gap: within the FrameSamp family the *where* trend is monotone (visual context $13.7\% \rightarrow$ separate expert $28.9\% \rightarrow$ action-side AdaLN 42.0%) and WeaveLA already sits at the peak's *where*; along *when*,

Table 1: **Left:** Trigger-source robustness on $\pi_{0.5}$+AP at the 16-task scale (success ↑, %, 50 episodes); columns share extractor + consumption, only the trigger differs (latent-shift = Stage 2 auxiliary re-used at deployment, no external supervision; oracle = simulator boundaries). Full per-task numbers in Appendix A.8. **Right:** Real-robot PICKXTIMES (N=3), single-task probe (n=20); setup in Appendix A.13, representative rollout in Figure 9).

Task / Slice	Weaver-off	Latent-shift	Oracle
STOPCUBE	2	2	**12**
PICKXTIMES	38	44	36
SWINGXTIMES (aggregate)	28	**46**	**46**
SWINGXTIMES (N=3 slice)	4	**21.7**	**30**
16-task aggregate	16.25	**17.4**	**17.0**

Condition	Success (%)
Weaver-off	45
Weaver-on	**70**

no-write and per-step append both fall below ours, per-keyframe retrieval underperforms because retrieved frames are not aligned with count events, and per-frame writes win only by paying a per-step cost. The residual gap therefore sits on the *what* axis (per-event count latent vs. dense per-frame buffer); the two strongest mechanisms share the same *where* (action-side modulation) and differ on *what*/*when*, suggesting **FrameSamp's dense buffer and WeaveLA's per-event latent are orthogonal** and could in principle be combined as parallel modulation streams (left to future work). Per-method numerical deltas and failure-mode analysis are in Appendix A.9.

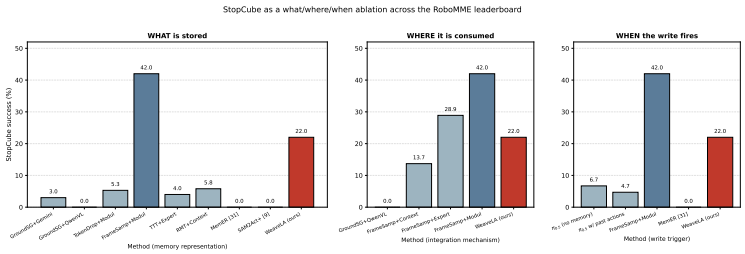

Figure 5: **STOPCUBE as a what/where/when ablation across the RoboMME baseline** (success ↑, %, 50 episodes). WeaveLA (red) is plotted against published MME-VLA variants and prior-method baselines. Each panel reorganises the same set of methods by one axis of the memory design.

4.4 Extractor design study: attention pooling vs. Q-Former

Replacing the single-step attention-pooling Weaver (Eq. 1) with a multi-layer decoder Q-Former [21] extractor (same trigger, same AdaRMS consumption, 16-task scale) collapses 16-task success from 23.3% to 3.5% ($\pi_{0.5}$+QF), with 56.3% of episodes terminating by rollout timeout rather than wrong action — attributable to higher inference cost and a less stable training trajectory (full numbers in Appendix A.7). We adopt attention pooling throughout.

4.5 Trigger source robustness: mechanism is decoupled from oracle

To verify that the simulator-provided sub-goal boundaries used in Sections 4.2–4.3 are not a load-bearing oracle, we fix the WeaveLA extractor and replace the trigger with a *model-internal* alternative: the Stage 2 contrastive auxiliary $\mathcal{L}_{\text{align}} + \mathcal{L}_{\text{ctr}}$ shapes the Weaver latent to align with sub-goal text embeddings, and we re-purpose this representation as a deployment-time event detector by firing a memory write when the projected latent's EMA cosine distance from the segment anchor exceeds a calibrated threshold (Appendix A.8). No external supervision is used at deployment.

The mechanism's lift replicates without oracle. The 16-task aggregate is indistinguishable (Table 1 left): latent-shift 17.4% vs oracle 17.0% vs no-memory 16.25%. Crucially, the mechanism-localisation signature of Figure 4 replicates exactly under latent-shift on SWINGXTIMES: N=1 saturates at 100%, and on the hardest slice (N=3) latent-shift rises from 4% to **21.7**%, recovering **68**% of the 4% → 30% oracle gap — *the contrastive auxiliary doubles as a deployable WHEN trigger at no extra training cost*. The one exception is STOPCUBE, whose boundary is purely

symbolic: latent-shift collapses to the no-memory baseline (2% vs 12% oracle); on the other 15 of 16 tasks, the mechanism is robust to trigger source.

Real-robot sanity check. On a physical PICKXTIMES task ("put the blue block into the plate" executed $N{=}3$ times per episode, six pick-and-place sub-goals; setup in Appendix A.13) with the same $\pi_{0.5}$+AP backbone and only the Weaver pathway toggled, Table 1 reports Weaver-off $9/20$ vs. Weaver-on $\mathbf{14/20}$ — direction consistent with the simulation pattern.

5 Conclusion and limitations

We presented **WeaveLA**, a cross-subtask latent memory channel that writes a compact 8-token latent at each sub-goal completion event and routes it directly into the next sub-task's action expert. On a $\pi_{0.5}$ backbone, gains are mechanism-aligned across three independent stratifications: the lift concentrates on the slice that requires the channel (SWINGXTIMES $N{=}3$: $0\% \rightarrow 47.8\%$) while $N{=}1$ stays unchanged, and the contribution is decoupled from simulator boundaries (a model-internal latent-shift trigger recovers 68% of the oracle gap without external supervision). **Limitations:** latent-shift collapses on count-critical STOPCUBE (2% vs 12% oracle), so symbol-counting tasks still need a learned symbolic detector; characterising the Q-Former instability (Section 4.4) and composing WeaveLA with input-side optimisations such as FrameSamp [8] are left to future work.

References

[1] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization, 2016. *arXiv preprint arXiv:1607.06450*.

[2] P.-L. Bacon, J. Harb, and D. Precup. The option-critic architecture. In *AAAI*, 2017.

[3] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, et al. PaliGemma: A versatile 3B VLM for transfer, 2024. *arXiv preprint arXiv:2407.07726*.

[4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, et al. π_0: A vision-language-action flow model for general robot control, 2024.

[5] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control, 2023.

[6] A. Bulatov, Y. Kuratov, and M. S. Burtsev. Recurrent memory transformer, 2022.

[7] M. Sobol Mark, J. Liang, M. Attarian, C. Fu, D. Dwibedi, D. Shah, and A. Kumar. BPP: Long-context robot imitation learning by focusing on key history frames, 2026. *arXiv preprint arXiv:2602.15010*.

[8] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.

[9] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman. Counting out time: Class-agnostic video repetition counting in the wild, 2020.

[10] H. Fang, M. Grotz, W. Pumacay, Y. R. Wang, D. Fox, R. Krishna, and J. Duan. SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation, 2025.

[11] Gemma Team. Gemma: Open models based on Gemini research and technology, 2024a. *arXiv preprint arXiv:2403.08295*.

[12] Gemma Team. Gemma 2: Improving open language models at a practical size, 2024b. *arXiv preprint arXiv:2408.00118*.

[13] S. Han, B. Qiu, Y. Liao, S. Huang, C. Gao, S. Yan, and S. Liu. RoboCerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation, 2025.

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, et al. LoRA: Low-rank adaptation of large language models, 2022.

[15] H. Hu, S. Dong, Y. Zhao, D. Lian, Z. Li, and S. Gao. TransRAC: Encoding multi-scale temporal correlation with transformers for repetitive action counting, 2022.

[16] H. Jang, S. Yu, H. Kwon, H. Jeon, Y. Seo, and J. Shin. ContextVLA: Vision-language-action model with amortized multi-frame context, 2025.

[17] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, et al. OpenVLA: An open-source vision-language-action model, 2024.

[18] T. Kipf, Y. Li, H. Dai, V. Zambaldi, A. Sanchez-Gonzalez, E. Grefenstette, P. Kohli, and P. Battaglia. CompILE: Compositional imitation learning and execution. In *ICML*, 2019.

[19] M. Koo, D. Choi, T. Kim, K. Lee, C. Kim, Y. Seo, and J. Shin. HAMLET: Switch your vision-language-action model into a history-aware policy, 2025. *arXiv preprint arXiv:2510.00695*.

[20] H. Li, F. Shen, D. Chen, L. Yang, X. Wang, J. Shi, Z. Bing, Z. Liu, and A. Knoll. ReMem-VLA: Empowering vision-language-action model with memory via dual-level recurrent queries, 2026. *arXiv preprint arXiv:2603.12942*.

[21] J. Li, D. Li, S. Savarese, and S. Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, 2023.

[22] R. Li, W. Guo, Z. Wu, C. Wang, H. Deng, Z. Weng, Y.-P. Tan, and Z. Wang. MAP-VLA: Memory-augmented prompting for vision-language-action model in robotic manipulation, 2025. *arXiv preprint arXiv:2511.09516*.

[23] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling, 2022.

[24] Y. Fan, P. Ding, S. Bai, X. Tong, Y. Zhu, et al. Long-VLA: Unleashing long-horizon capability of vision-language-action model for robot manipulation. In *CoRL*, 2025.

[25] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, et al. GR00T N1: An open foundation model for generalist humanoid robots, 2025.

[26] NVIDIA GEAR Team. GR00T N1.6: An improved open foundation model for generalist humanoid robots. `https://research.nvidia.com/labs/gear/gr00t-n1_6`, 2025.

[27] R. Paiss, A. Ephrat, O. Tov, S. Zada, I. Mosseri, M. Irani, and T. Dekel. Teaching CLIP to count to ten, 2023.

[28] Z. Liu, Y. Wang, S. Zheng, T. Pan, L. Liang, Y. Fu, and X. Xue. ReasonGrounder: LVLM-guided hierarchical feature splatting for open-vocabulary 3D visual grounding and reasoning. In *CVPR*, 2025a.

[29] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization, 2025.

[30] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, et al. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation, 2025.

[31] S. Sinha, A. Stergiou, and D. Damen. Every shot counts: Using exemplars for repetition counting in videos. In *ACCV*, 2024.

[32] A. Sridhar, J. Pan, S. Sharma, and C. Finn. MemER: Scaling up memory for robot control via experience retrieval, 2025. *arXiv preprint arXiv:2510.20328*.

[33] J. Su, A. Murtadha, Y. Lu, S. Pan, Wen. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[34] Z. Liu, S. Zheng, S. Chen, C. Zhao, L. Liang, X. Xue, and Y. Fu. A neural representation framework with LLM-driven spatial reasoning for open-vocabulary 3D visual grounding. In *ACM MM*, 2025b.

[35] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

[36] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, et al. MEM: Multi-scale embodied memory for vision language action models, 2026.

[37] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith. Using reward machines for high-level task specification and decomposition in reinforcement learning. In *ICML*, 2018.

[38] P. Vaezipoor, A. C. Li, R. Toro Icarte, and S. A. McIlraith. LTL2Action: Generalizing LTL instructions for multi-task RL. In *ICML*, 2021.

[39] Z. Liu, Y. Wang, K. Wang, L. Liang, X. Xue, and Y. Fu. Spatial-temporal aware visuomotor diffusion policy learning. In *ICCV*, 2025c.

[40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[41] X. Guo, Z. Huang, Z. Shi, Z. Song, and J. Zhang. Your vision-language model can't even count to 20: Exposing the failures of VLMs in compositional counting, 2025. *arXiv preprint arXiv:2510.04401*.

[42] J. Zhang, Y. Guo, X. Chen, Y.-J. Wang, Y. Hu, C. Shi, and J. Chen. HiRT: Enhancing robotic control with hierarchical robot transformers. In *CoRL*, 2024.

[43] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies, 2024.

[44] Z. Liu, Y. Gu, S. Zheng, Y. Fu, X. Xue and Y. Jiang. TriVLA: A triple-system-based unified vision-language-action model for general robot control, 2025d. *arXiv preprint arXiv:2507.01424*.

[45] Z. Liu, Y. Gu, Y. Wang, X. Xue, and Y. Fu. ActiveVLA: Injecting active perception into vision-language-action models for precise 3D robotic manipulation, 2026. *arXiv preprint arXiv:2601.08325*.

[46] G. Zhang, M. Fu, and S. Yan. MemGen: Weaving generative latent memory for self-evolving agents, 2025. *arXiv preprint arXiv:2509.24704*.

[47] B. Zhang and R. Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.

[48] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023a.

[49] Y. Chen, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen. Non-Markovian long-horizon robot manipulation via keyframe chaining, 2026. *arXiv preprint arXiv:2603.01465*.

[50] H. Wu, T. Chen, J. Wang, X. Li, and L. Fang. StreamVLA: Breaking the reason-act cycle via completion-state gating, 2026. *arXiv preprint arXiv:2602.01100*.

[51] R. Yang, Z. An, L. Zhou, and Y. Feng. SeqVLA: Sequential task execution for long-horizon manipulation with completion-aware vision-language-action model, 2025. *arXiv preprint arXiv:2509.14138*.

[52] W. Wan, Y. Zhu, R. Shah, and Y. Zhu. LOTUS: Continual imitation learning for robot manipulation through unsupervised skill discovery. In *ICRA*, 2024.

[53] Y. Zhu, P. Stone, and Y. Zhu. Bottom-up skill discovery from unsegmented demonstrations for long-horizon robot manipulation. *IEEE Robotics and Automation Letters*, 2022.

A Technical Appendices and Supplementary Material

A.1 Full 16-Task RoboMME Breakdown

Tasks fall into three groups: *repetition*, where the gain concentrates; *memory-of-context*, where the effect is mixed and depends on whether the task semantics require carrying information forward; and *other* / largely zero-shot at the 6-task scale. The aggregate 16-task improvement is concentrated on the repetition / dependent-stop tasks.

A.2 Aggregate Per-Task Numerical Table (referenced by Figure 3)

Table 3 provides the per-task numerical values that the bar chart in Figure 3 renders. The "Weaver off (no-weaving baseline)" rows use our framework with the memory-weaving pathway disabled (no Weaver module, no AdaRMS memory injection); they share training data, optimiser, and step counts with the corresponding Weaver-on rows. Improvements concentrate on SWINGXTIMES and STOPCUBE at both training scales.

Table 2: Full RoboMME 16-task success rates (↑, %, 50 episodes per task) for the four $\pi_{0.5}$+AP configurations. The 6-task training rows leave non-trained tasks at zero-shot evaluation for completeness. Bold entries mark the two repetition tasks (PICKXTIMES, SWINGXTIMES) and the dependent-stop task (STOPCUBE) where the cross-subtask channel is expected to engage.

Task	W-on (6)	W-off (6)	W-on (16)	W-off (16)
Repetition (gain expected)				
PICKXTIMES	40.0	40.0	34.0	38.0
SWINGXTIMES	56.0	32.0	50.0	28.0
Memory-of-context				
STOPCUBE	22.0	8.0	12.0	2.0
BUTTONUNMASK	18.0	24.0	28.0	16.0
BUTTONUNMASKSWAP	0.0	0.0	4.0	8.0
VIDEOUNMASK	26.0	26.0	24.0	26.0
VIDEOUNMASKSWAP	18.0	22.0	22.0	20.0
Other / largely zero-shot at 6-task				
VIDEOPLACEBUTTON	32.0	30.0	22.0	32.0
VIDEOPLACEORDER	26.0	26.0	14.0	18.0
PICKHIGHLIGHT	12.0	10.0	12.0	12.0
MOVECUBE	18.0	18.0	16.0	18.0
BINFILL	0.0	0.0	28.0	24.0
PATTERNLOCK	0.0	0.0	8.0	8.0
INSERTPEG	0.0	0.0	2.0	2.0
ROUTESTICK	0.0	2.0	12.0	6.0
VIDEOREPICK	0.0	0.0	2.0	2.0
Overall (16 tasks)	16.75	14.88	18.12	16.25

Table 3: Numerical version of Figure 3: RoboMME six trained tasks, $\pi_{0.5}$+AP. Success rate (↑, %, 50 episodes per task).

Method	PickX	BtnUM	BtnUMSw	PickHl	StopC	SwingX	Avg.
$\pi_{0.5}$ official [8]	42.9	22.2	6.7	11.3	6.7	35.6	20.9
$\pi_{0.5}$+AP, 6-task training							
Weaver off (no-weaving baseline)	40.0	24.0	0.0	10.0	8.0	32.0	19.0
Weaver on	40.0	18.0	0.0	12.0	**22.0**	**56.0**	**24.7**
$\pi_{0.5}$+AP, 16-task training							
Weaver off (no-weaving baseline)	38.0	16.0	8.0	12.0	2.0	28.0	17.3
Weaver on	34.0	28.0	4.0	12.0	**12.0**	**50.0**	**23.3**

A.3 N-Stratified Numerical Table (referenced by Figure 4)

Table 4 provides the per-N numerical values that the line chart in Figure 4 renders. N is the instruction-encoded number of sub-goals to be executed in the episode. The gap opens precisely at $N \geq 2$, where the next stage depends on what was just completed. SWINGXTIMES $N=3$ at the 6-task scale moves from $0/23$ to $11/23$ ($0\% \rightarrow 47.8\%$).

A.4 Difficulty-Stratified Analysis

Table 5 reports the difficulty-stratified success rates referenced in Section 4.3. Improvements concentrate on Hard episodes ($1.4\% \rightarrow 12.5\%$); Easy and Medium episodes show only marginal changes and are omitted.

A.5 Per-Task Paired Contingency

Table 6 reports the full per-task paired contingency referenced in Section 4.3. For each task at the 6-task training scale, we count both-succeed (BB), Weaver-on-only (W-on), Weaver-off-only (W-off), and neither (NN) over 50 matched episode pairs.

Table 4: Numerical version of Figure 4: RoboMME success on the two repetition tasks, stratified by instruction-encoded sub-goal count N (succ./total, %).

	6-task training		16-task training	
Subset	W on	W off	W on	W off
SWINGXTIMES				
$N = 1$	15/15 (100)	15/15 (100)	14/15 (93)	13/15 (87)
$N = 2$	2/12 (17)	1/12 (8)	4/12 (33)	0/12 (0)
$N = 3$	**11/23 (47.8)**	0/23 (0)	7/23 (30)	1/23 (4)
PICKXTIMES				
$N = 1$	16/16 (100)	16/16 (100)	16/16 (100)	16/16 (100)
$N = 2$	3/10 (30)	2/10 (20)	1/10 (10)	1/10 (10)
Pooled across both repetition tasks				
$N = 1$	100%	100%	96.8%	93.5%
$N \geq 2$	**24.6%**	7.2%	**17.4%**	5.8%

Table 5: RoboMME difficulty-stratified success on the six trained tasks (6-task training, Hard sub-pool).

Variant	Hard
Weaver on	**12.5%**
Weaver off	1.4%

A.6 Per-Episode Comparison: Weaver-on vs. Weaver-off

Figure 6 provides a side-by-side rollout comparison of the Weaver module on two timing- and counting-critical RoboMME tasks, with each row showing 9 key-event frames from a single rollout. Every cell preserves both the external camera (left half) and the wrist camera (right half), so the global trajectory and the fine-grained grasp can be inspected together. The SWINGXTIMES task (top two rows, episode 3) requires the policy to pick up the green cube and shuttle it back and forth between the right-side and left-side targets, completing three full repetitions before pressing the stop button. The STOPCUBE task (bottom two rows, episode 10) requires the policy to watch a cube that automatically oscillates over a target and press the stop button at the exact moment the cube reaches the target for the third time. Rows 1 and 3 are the Weaver-off rollouts of these episodes, and both fail: in SWINGXTIMES the policy overshoots and performs 3.5 swings (an extra half cycle past the third target visit) before pressing the button, so the episode is judged a failure; in STOPCUBE the policy presses the stop button right after the cube passes the target for the *second* time rather than the third, so the episode is also marked a failure. Rows 2 and 4 are the Weaver-on rollouts of the *same* episodes: the model correctly completes exactly three swings before pressing in SWINGXTIMES, and triggers the press at exactly the third target crossing in STOPCUBE so the cube settles directly on the bullseye. Because the initial state and episode index are identical across each pair, the consistent success-with-Weaver / failure-without-Weaver outcome isolates the contribution of the module, indicating that Weaver is essential for sub-task counting and for identifying temporally critical moments in tasks.

A.7 Q-Former Extractor: Full Per-Task Breakdown

Table 7 reports the full 16-task per-task success rates for $\pi_{0.5}$+QF (Q-Former extractor) at the 16-task training scale, against the matched $\pi_{0.5}$+AP no-memory baseline (Weaver off, 16-task), so that all three runs share the same backbone, training data, and step counts. The Q-Former aggregate is 3.5%, versus 16.25% for the no-memory baseline and 23.3% for $\pi_{0.5}$+AP at the same scale. The dominant failure mode for $\pi_{0.5}$+QF is rollout timeout: $450/800$ (56.3%) episodes terminated by the per-episode time budget rather than by an incorrect action, compared with negligible timeout rates for $\pi_{0.5}$+AP. The Q-Former extractor's combination of higher inference cost per memory write and a less stable training trajectory leaves the action expert with insufficient time to commit a complete action chunk before the episode times out.

Table 6: Per-episode paired contingency at the 6-task training scale across all 16 RoboMME tasks (800 episode pairs). The W-on / W-off only-success imbalance is concentrated on SWINGXTIMES and STOPCUBE; non-repetition / non-dependent-stop tasks are within paired-noise range.

Task	BB	W-on only	W-off only	NN	Net Δ
SWINGXTIMES	15	**13**	1	21	+12
STOPCUBE	3	**8**	1	38	+7
PICKXTIMES	16	4	4	26	0
VIDEOPLACEBUTTON	10	6	5	29	+1
PICKHIGHLIGHT	2	4	3	41	+1
VIDEOPLACEORDER	11	2	2	35	0
VIDEOUNMASK	12	1	1	36	0
MOVECUBE	8	1	1	40	0
BUTTONUNMASK	7	2	5	36	−3
VIDEOUNMASKSWAP	8	1	3	38	−2
Other (6 tasks)	0	0	1	299	−1

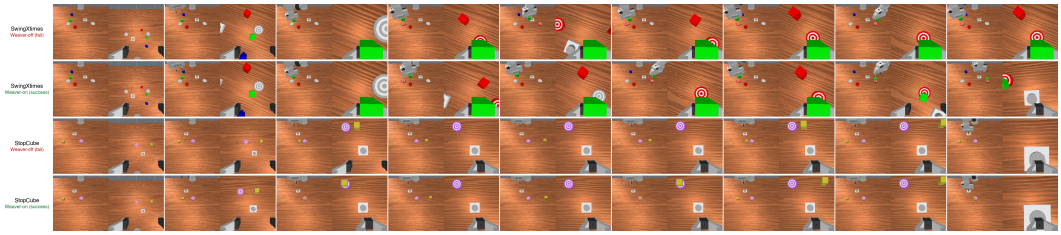

Figure 6: **Per-frame comparison of Weaver-on vs. Weaver-off on two timing-critical tasks under identical initial conditions.** Top two rows: SWINGXTIMES — pick up the green cube, swing it between the right and left targets three times, then press the stop button. Bottom two rows: STOPCUBE — press the button at the exact moment the oscillating cube reaches the target for the third time. Without Weaver (rows 1 and 3), the policy miscounts the swing repetitions and mistimes the button press; with Weaver enabled (rows 2 and 4), the same episodes are solved successfully.

A.8 Trigger Source Robustness: Full 16-Task Breakdown

Table 8 reports per-task success on all 16 RoboMME tasks for the trigger-source robustness study in Section 4.5. All three columns share the same $\pi_{0.5}$+AP backbone at the 16-task training scale; only the trigger source differs. Latent-shift derives the trigger from the Stage 2 contrastive auxiliary at deployment with no external supervision; Oracle uses simulator-provided sub-goal boundaries; Weaver-off disables the memory pathway entirely.

A.9 StopCube what/where/when Ablation Against RoboMME Baselines

STOPCUBE is the canonical *count, time-critical* task in RoboMME [8]: the policy must press a button at exactly the N-th moment a moving cube reaches the target, requiring both event counting and precise temporal coordination. The RoboMME paper itself reports that the symbolic upper bound (GROUNDSG+ORACLE, an oracle subgoal planner with the policy still doing low-level control) only reaches 49.7% on this task, indicating that *precise visuomotor control becomes the primary bottleneck* on STOPCUBE. Table 9 compares WeaveLA against the published RoboMME baseline pool as a what/where/when ablation, drawing all baseline numbers from Table 12 of Dai et al. [8]; WeaveLA's two rows are our $\pi_{0.5}$+AP runs.

Where WeaveLA stands. WeaveLA's 22.0% at the 6-task scale and 12.0% at the 16-task scale outperform 12 of the 14 published MME-VLA variants in the baseline pool, both prior-method baselines (MemER, SAM2Act+, both at 0.0%), the no-memory $\pi_{0.5}$ baseline (6.7%), and the past-actions variant (4.7%). FrameSamp+Modul remains the strongest published number at 42.0%. We discuss below (i) why MemER and SAM2Act+ fail at 0.0% on this specific task, and (ii) why WeaveLA's mechanism is orthogonal to FrameSamp+Modul and can be combined with it.

Table 7: $\pi_{0.5}$+QF (Q-Former extractor) vs. the matched $\pi_{0.5}$+AP no-memory baseline (Weaver off, 16-task). Success rates (↑, %, 50 episodes per task). $\pi_{0.5}$+QF underperforms sharply on every repetition / dependent-stop task and times out on the majority of long-horizon episodes.

Task	$\pi_{0.5}$+AP no-memory (16-task)	$\pi_{0.5}$+QF (16-task)
SWINGXTIMES	28.0	0
PICKXTIMES	38.0	0
STOPCUBE	2.0	0
BUTTONUNMASK	16.0	0
BUTTONUNMASKSWAP	8.0	0
PICKHIGHLIGHT	12.0	2
VIDEOUNMASK	26.0	24
VIDEOUNMASKSWAP	20.0	16
VIDEOPLACEBUTTON	32.0	–
VIDEOPLACEORDER	18.0	–
MOVECUBE	18.0	8
BINFILL	24.0	–
PATTERNLOCK	8.0	4
INSERTPEG	2.0	–
ROUTESTICK	6.0	2
VIDEOREPICK	2.0	0
Overall (16 tasks)	16.25	3.5
Timeout rate	–	56.3%

Table 8: Per-task success rates (↑, %, 50 episodes per task) for the trigger-source robustness study, on all 16 RoboMME tasks at the $\pi_{0.5}$+AP / 16-task training scale.

Task	Weaver-off	Latent-shift	Oracle
BINFILL	24	26	20
STOPCUBE	2	2	12
PICKXTIMES	38	44	36
SWINGXTIMES	28	46	46
BUTTONUNMASK	16	14	14
VIDEOUNMASK	26	28	24
VIDEOUNMASKSWAP	20	14	20
BUTTONUNMASKSWAP	8	4	8
PICKHIGHLIGHT	12	10	6
VIDEOREPICK	2	4	0
VIDEOPLACEBUTTON	32	26	20
VIDEOPLACEORDER	18	18	22
MOVECUBE	18	22	16
INSERTPEG	2	4	4
PATTERNLOCK	8	8	8
ROUTESTICK	6	8	8
16-task aggregate	16.25	17.4	17.0
SWINGXTIMES N=3 slice	4	21.7	30

Why MemER fails at STOPCUBE. MemER [32] couples a VLM that retrieves keyframes from past observations with a symbolic-subgoal policy that executes the retrieved instruction. This is well-matched to *dynamic scene-change* tasks where the value of memory is recovering an object identity from a previous frame — and indeed the RoboMME paper notes MemER excels on those tasks. STOPCUBE is a different problem: the cube's position relative to the target evolves continuously between visits, the policy must commit a discrete press action at a specific frame, and there is no past keyframe whose retrieval would tell the policy when to press *now*. The keyframe-retrieval inductive bias does not produce the temporal precision the task requires. Empirically the model scores $0/50$.

Why SAM2Act+ fails at STOPCUBE. SAM2Act+ [10] predicts *discrete waypoint actions* that are then executed by an external motion planner. The RoboMME paper makes the failure mode explicit: this design *"degrades on tasks requiring precise continuous control such as* STOPCUBE

Table 9: StopCube success rates (↑, %, 50 episodes per task) as a what/where/when ablation against the RoboMME baseline pool. Baseline numbers are taken from Table 12 of Dai et al. [8] (also published at `https://robomme.github.io/leaderboard.html`). Methods are grouped by family. WeaveLA outperforms 12 of the 14 published MME-VLA variants and the two prior-method baselines (MemER, SAM2Act+); FrameSamp+Modul remains stronger. Bold marks the best non-oracle entries.

Family	Method	StopCube
Oracle / upper bounds		
Human study	Human (oracle planner)	90.0
Symbolic + oracle	GroundSG+Oracle	49.7
Prior memory-augmented baselines		
Perceptual + symbolic	MemER [32]	0.0
Perceptual + waypoint	SAM2Act+ [10]	0.0
No-memory / minimal-memory $\pi_{0.5}$ baselines		
No memory	$\pi_{0.5}$ [29]	6.7
Past actions concatenated	$\pi_{0.5}$ w/ past actions	4.7
Symbolic memory (non-oracle subgoal predictors)		
Symbolic + Gemini	SimpleSG+Gemini	2.0
Symbolic + Gemini	GroundSG+Gemini	3.0
Symbolic + QwenVL	SimpleSG+QwenVL	0.4
Symbolic + QwenVL	GroundSG+QwenVL	0.0
Recurrent memory		
Test-time training	TTT+Context / Modul / Expert	3.3 / 2.1 / 4.0
Recurrent memory transformer	RMT+Context / Modul / Expert	5.8 / 4.7 / 5.6
Perceptual memory (token-level)		
Token-drop	TokenDrop+Context / Modul / Expert	3.1 / 5.3 / 4.2
Perceptual memory (frame-level, dense)		
Frame-sampling	FrameSamp+Context	13.7
Frame-sampling	FrameSamp+Expert	28.9
Frame-sampling	**FrameSamp+Modul**	**42.0**
This work — per-event latent memory		
Sub-goal-event latent	WeaveLA ($\pi_{0.5}$+AP, 16-task)	12.0
Sub-goal-event latent	**WeaveLA ($\pi_{0.5}$+AP, 6-task)**	**22.0**

and InsertPeg.*"* A waypoint planner cannot represent "press the button at exactly this moment in the cube's trajectory" — the action space itself is the wrong granularity. Empirically the model also scores $0/50$.

Why FrameSamp+Modul wins, and why WeaveLA is orthogonal to it. FrameSamp+Modul reaches 42.0% by routing a wide perceptual buffer of past frames into the action expert via AdaLN modulation. Continuous awareness of the cube's distance from the target across many recent frames is directly useful for time-sensitive control; the RoboMME paper specifically attributes its strength on time-sensitive and motion-centric tasks to this dense perceptual buffer. WeaveLA's mechanism is structurally different along the *when*-axis: instead of a dense per-frame buffer, the Weaver writes a single compact 8-token latent at each completed sub-goal event — the natural unit at which the count N is incremented. Crucially, the two are orthogonal: a dense perceptual buffer answers *"how close is the cube to the target right now"* while a per-event latent answers *"which visit are we currently on"*. They occupy different design axes (*what* is stored and *when* the write fires) and consume the action expert through compatible interfaces (both are AdaLN/AdaRMS modulation in the same Gemma backbone). In principle, the FrameSamp dense buffer and the WeaveLA per-event latent could be concatenated as parallel modulation streams without any architectural conflict; we leave this combination to future work.

Why WeaveLA already beats other strong baselines. Among methods that do not use a dense perceptual buffer, WeaveLA is the strongest published number on StopCube. The TokenDrop variants (3.1–5.3%) drop visual tokens by RGB difference, removing exactly the slow continuous

evidence the task needs; the recurrent memory variants (TTT 2.1–4.0%, RMT 4.7–5.8%) compress history into a fixed latent state without any explicit alignment to sub-goal events, so the count signal is diluted into the rest of the recurrent state; and the symbolic-subgoal variants with non-oracle subgoal predictors (Gemini, QwenVL) score 0.0–3.0% because subgoal prediction errors compound across visits. WeaveLA's sub-goal-event trigger (Section 3.3) avoids these failure modes: the write fires exactly at each visit, and the latent dimensionality is allocated to that single event rather than diffused across a recurrent state.

A.10 Bottleneck Width Ablation: 4 vs. 8 vs. 16 Latents

Table 10 reports per-task success for $\pi_{0.5}$+AP on the 16-task training scale across three memory bottleneck widths — $N{=}4$, $N{=}8$ (the main configuration), and $N{=}16$ — all run under the same Stage 2 training schedule. The picture across $N \in \{4, 8, 16\}$ is non-monotonic only in one direction: there is a binding floor below which the bottleneck collapses ($N{=}4$ aggregates to 2.0%, with $0/50$ on every repetition / dependent-stop task and residual competence only on a handful of memory-of-context tasks), 8 tokens already saturate the channel, and increasing capacity to 16 tokens does not improve aggregate success (17.5% at $N{=}16$ vs. 23.3% at $N{=}8$). SWINGXTIMES success is the same at 8 and 16 tokens (50.0%), while every other task either matches or trails the 8-token configuration.

Table 10: Memory bottleneck width ablation on $\pi_{0.5}$+AP, 16-task training. Both runs use the same Stage 2 training schedule; only the bottleneck width N differs. Increasing or decreasing the bottleneck width does not improve aggregate success.

Task	4 latents	8 latents (main)	16 latents
SWINGXTIMES	0.0	50.0	50.0
PICKXTIMES	0.0	34.0	36.0
STOPCUBE	0.0	12.0	8.0
BUTTONUNMASK	0.0	28.0	20.0
BUTTONUNMASKSWAP	0.0	4.0	10.0
PICKHIGHLIGHT	0.0	12.0	6.0
VIDEOUNMASK	16.0	24.0	22.0
VIDEOUNMASKSWAP	12.0	22.0	16.0
VIDEOPLACEBUTTON	0.0	22.0	26.0
VIDEOPLACEORDER	0.0	14.0	22.0
MOVECUBE	2.0	16.0	24.0
BINFILL	0.0	28.0	24.0
PATTERNLOCK	2.0	8.0	10.0
INSERTPEG	0.0	2.0	2.0
ROUTESTICK	0.0	12.0	4.0
VIDEOREPICK	0.0	2.0	0.0
Overall (16 tasks)	2.0	18.12	17.50

A.11 Staged Training and the `weaver_latent_norm` Collapse

Training proceeds in three stages, separating action grounding, memory warmup, and full integration; see Section 3.5 for the high-level rationale. Here we provide the diagnostic that motivated the staged design.

We define `weaver_latent_norm` as the ℓ_2 norm of the pooled Weaver latent before semantic normalisation:

$$\texttt{weaver_latent_norm} \;=\; \mathbb{E}_B\big[\,\big\|m_{k-1}\big\|_2\,\big]\,, \tag{5}$$

where m_{k-1} is the output of Eq. 1. While this metric does not directly measure semantic correctness, it reflects whether the raw memory pathway maintains a stable signal for action modulation. In our model the action expert consumes the raw Weaver latent (via AdaRMS), while the alignment and contrastive losses operate on a separately normalised projection. As a consequence, semantic auxiliary losses can remain low even when the raw latent magnitude degenerates, masking a hidden representation collapse.

We compared two training strategies on the 16-task scale: (i) the staged strategy described in Section 3.5, and (ii) a single merged stage that combines all three objectives from the begin-

ning. Both strategies kept action and alignment losses stable. However, in the merged-stage run, `weaver_latent_norm` decayed monotonically across training and downstream evaluation success collapsed to near zero. In the staged run, `weaver_latent_norm` remained stable throughout. Figure 7 shows the diagnostic curves.

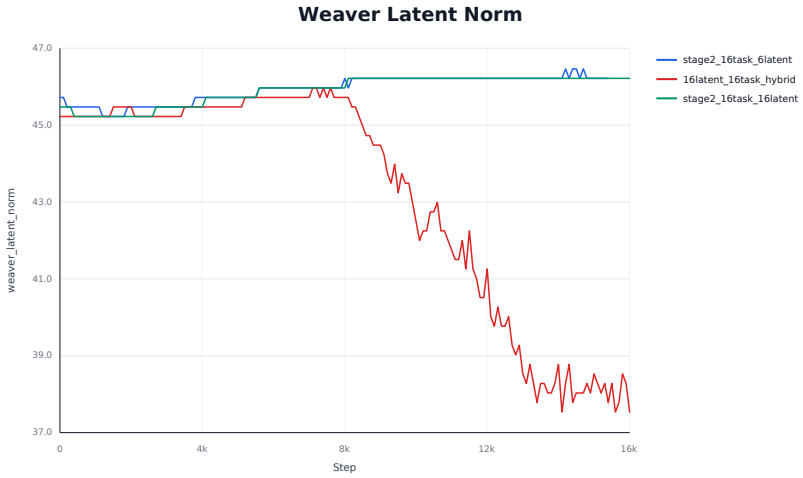

Figure 7: `weaver_latent_norm` during training, 16-task scale. The two staged-training runs (`stage2_16task`, `stage2_16task_from_stage1`) keep the latent norm stable at ~ 46 throughout training. The merged-stage run (`16task_hybrid`) shows monotonic decay starting around step 8k, ending near ~ 38 at step 16k. Downstream evaluation success on the merged-stage run collapsed to near zero, despite stable action and alignment loss curves — a hidden representation collapse not visible from the loss alone.

A.12 Hyperparameters and Compute

$\pi_{0.5}$+AP training schedule. All four $\pi_{0.5}$+AP rows (Weaver-on/off $\times$ 6-task/16-task) share the same training schedule: Stage 0 task-set adaptation for 40k steps, Stage 1 memory warmup for 8k steps with K=2, and Stage 2 full Weaver training for 16k steps with variable $K \in \{2, 3, 4\}$. The model uses an action horizon of 20 with `num_action_steps`=8. Trainable parameters include the PaliGemma LoRA adapters (rank 128, α=128 on the `gemma_2b_lora` variant), the Weaver query latents and projections, and the action-side memory projection that feeds AdaRMS. Optimisation uses AdamW with separate learning rates: base 1×10^{-5}, LoRA 5×10^{-5}, memory and modulation 1×10^{-4}. Stage 2 uses λ_{align}=0.05 and λ_{ctr}=0.02.

Compute. All training was performed on NVIDIA A100 80GB GPUs with FSDP. The total $\pi_{0.5}$+AP training pipeline (Stage 0 + Stage 1 + Stage 2) takes approximately 191 A100-80GB GPU-hours, broken down as Stage 0 task-set adaptation 36.9 GPU-hours, Stage 1 memory warmup 10.5 GPU-hours, and Stage 2 full Weaver training 143.6 GPU-hours. The 16-latent bottleneck-width ablation (Appendix A.10) consumes 140.8 GPU-hours on its Stage 2 run under the same schedule. Inference rollouts on the 800-episode RoboMME evaluation suite take approximately 40 GPU-hours per condition on a single A100. The Q-Former extractor variant trains in approximately the same wall-clock time as $\pi_{0.5}$+AP.

Trainable parameter count. The Weaver module trained on top of $\pi_{0.5}$ adds approximately 32M memory-pathway parameters (Weaver query latents, attention-pooling projections, AdaRMS conditioning) and 14M LoRA-adapter parameters on top of the frozen ~ 3.4B base policy — about 1.4% of the base model. Because the memory write is triggered only at sub-goal boundaries rather than every frame, the runtime cost of the memory channel during rollout is negligible compared to dense per-frame memory mechanisms.

A.13 Real-Robot Setup

The real-robot PICKXTIMES experiment of Section 4.5 runs on a PiPER 6-DoF arm with a parallel-jaw gripper. The workspace is a black-cloth tabletop with a green backdrop holding three coloured cubes (blue, red, green), a white plate, and a target disc. Two cameras feed the policy: a wrist-mounted RGB camera and a third-person RGB camera; an additional Intel RealSense top-down camera is used only for monitoring. At the start of each of the 20 episodes per condition the cubes and plate are reset to randomised positions inside the workspace and the policy is issued the prompt "put the blue block into the plate"; an episode is scored as a success if the blue block is in the plate after $N{=}3$ pick-and-place repetitions.

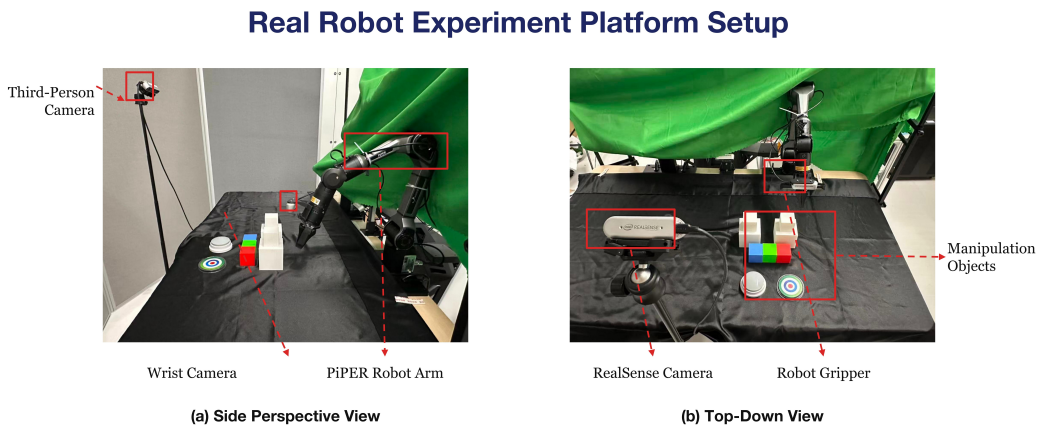

Figure 8: **Real-robot PICKXTIMES platform.** (a) Side perspective view, showing the PiPER arm, the third-person camera, the wrist camera, and the manipulation objects on the workspace. (b) Top-down view, showing the Intel RealSense monitoring camera, the gripper, and the layout of the three coloured blocks and the white plate.

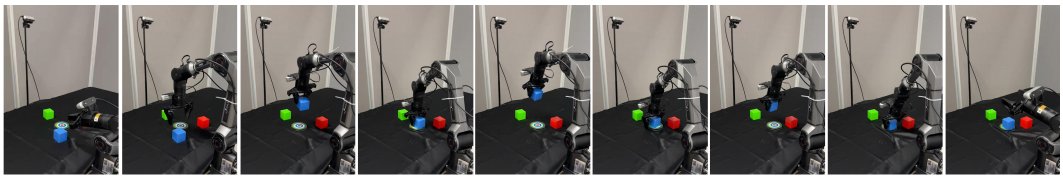

Figure 9: **PICKXTIMES task example.** In this instance, the goal is to pick up the blue cube and place it on the target, repeating this pick-and-place action three times, then return to default to stop. Frames are sampled at sub-goal completion events from a successful $N{=}3$ rollout, illustrating the three pick–place repetitions and the return-to-default termination.

A.14 Sub-Goal Boundary Annotation Tool

Sub-goal completion frames for the real-robot demonstrations are produced with a lightweight web-based annotator that synchronises all four camera streams (`cam_high`, `cam_head`, `cam_left_wrist`, `cam_right_wrist`) and lets the annotator step through frames at adjustable speed and mark / unmark the frame index of each sub-goal completion event. The marked indices are saved per episode (e.g. `[30, 54, 88]` for a 3-sub-goal episode) and consumed by the Stage 2 training pipeline as the WHEN trigger supervision. Figure 10 shows the annotator interface.

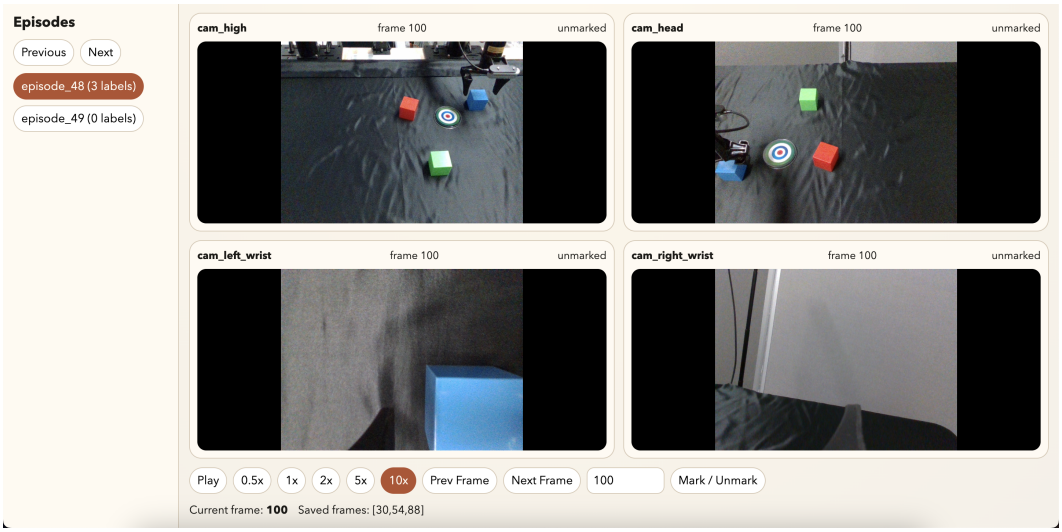

Figure 10: **Web-based sub-goal boundary annotator.** The four camera streams are displayed synchronously; the annotator scrubs through frames using the speed buttons (`0.5x`–`10x`) or per-frame controls and presses `Mark / Unmark` at each sub-goal completion event. The current frame index and the list of saved frame indices for the active episode are shown below the playback controls; an episode is finalised once all sub-goal boundaries are marked.